# Not Ready Yet: An Industrial Assessment of Coding Agents for Feature Implementation and the Promise of Harness Engineering

Zihan Wu*
Department of Electrical Engineering, City University of Hong Kong

Yun Peng*,†
Department of Computer Science and Engineering, The Chinese University of Hong Kong
ypeng@cse.cuhk.edu.hk

Chun Yong Chong
School of Information Technology, Monash University Malaysia

Xin Zhou
Singapore Management University

Zeyang Zhuang
The Chinese University of Hong Kong

Rui Shu
Independent Researcher

Kui Liu
Huawei Technologies Co., Ltd.

## Abstract

LLM-based coding agents now solve a large fraction of single-file bug-fix benchmarks, prompting industry interest in deploying them for multi-file feature implementation. Yet no rigorous, multi-repository benchmark exists for this setting. We evaluate four state-of-the-art coding agents—Claude Opus 4.6, GPT-5.4 (Codex), Composer-2 (Cursor Agent), and GLM-5.1 (OpenCode)—on 62 real feature-implementation tasks drawn from production repositories: 12 industrial tasks from Huawei CANN, MindSpeed, torch_npu, and the upstream Kubernetes project, plus 50 open-source tasks (CapBench) spanning Kafka, CPython, Airflow, TensorFlow, Django, and 10 other well-known projects in C, C++, Go, Java, and Python. Each of the $4 \times 62 \times 2 = 496$ task–agent–prompt combinations is independently scored by four LLM judges using a three-dimension rubric (functional correctness, completeness, behavioral equivalence), combined by majority vote with a conservative tie-break. Across 496 experiments, only 11 PASS (2.2%); the best agent (Cursor Composer-2) passes 3.2% of runs and the weakest (OpenCode GLM-5.1) 0.8%. Three patterns emerge: (i) an absolute complexity cliff—0% PASS on any High-complexity task for any agent; (ii) an OS-vs-framework cliff—Kubernetes refactors yield 0% PASS across 32 experiments while smaller CANN tasks retain 17.5% PASS; (iii) prompt specificity is the single strongest lever for top agents (Codex/Claude jump from 0% on short prompts to 4.8% on long prompts), while it saturates for weaker agents. The results indicate that current coding agents are not ready for autonomous multi-file feature implementation; their production use requires either prompt-scaffolded human oversight or restricting scope to small, self-contained tasks.

**ACM Reference Format:**
Zihan Wu*, Yun Peng*,†, Chun Yong Chong, Xin Zhou, Zeyang Zhuang, Rui Shu, and Kui Liu. 2026. Not Ready Yet: An Industrial Assessment of Coding Agents for Feature Implementation and the Promise of Harness Engineering. In . ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Software engineering teams are rapidly integrating AI into their development workflows. Industry surveys report that over 80% of professional developers now use AI-assisted coding tools [6], and frontier coding agents—autonomous systems that navigate codebases, write code, and run tests—have demonstrated remarkable progress on established benchmarks. On SWE-bench Verified, the most widely cited evaluation for coding agents, top systems now resolve over 70% of tasks [7]. These results have generated significant enthusiasm for deploying coding agents on increasingly ambitious engineering tasks, including full feature implementation in enterprise codebases.

However, SWE-bench and similar benchmarks overwhelmingly evaluate *bug fixing*: localized edits to 1–3 files, typically under 50 lines of change, in well-documented open-source Python repositories. Feature implementation in industrial settings is a fundamentally different class of task. It requires API design, feature gate lifecycle management, cross-module coordination, backward compatibility constraints, test suite construction, and adherence to project-specific conventions—engineering activities that demand architectural reasoning, not just code generation. A Kubernetes feature gate, for example, touches API types, validation, defaulting, conversion, controllers, and dozens of generated files across 50–100 files; a CANN operator implementation requires host code, device kernels, tiling engines, and framework integration layers spanning C++, AscendC, and Python. No systematic evaluation exists for how coding agents perform on this class of tasks in production codebases.

*Equal contribution. †Corresponding author: ypeng@cse.cuhk.edu.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*Conference'17, Washington, DC, USA*
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

We present the first industrial-scale capability assessment of coding agents on feature implementation tasks. Our benchmark comprises 62 tasks: 12 industrial tasks drawn from four Huawei production repositories—Kubernetes (Go), CANN operator kernels (C++/AscendC), MindSpeed training framework (Python), and torch_npu (Python)—spanning three complexity tiers from single-file fixes to 98-file architectural changes, plus 50 multi-file tasks from the CapBench open-source suite (Kafka, Airflow, CPython, Django, Spring, and others). We evaluate four commercially available agent configurations across 496 experiments—each independently scored by four LLM evaluators (Claude, Codex, Cursor, GLM) for cross-validation via majority-vote verdicts with conservative tie-breaking—and find a clear answer: **no.** The best configuration (Cursor Composer-2) achieves only 3.2% PASS; Claude Opus 4.6 and GPT-5.4 (Codex) tie at 2.4%; OpenCode GLM-5.1 reaches 0.8%. Overall, only 11 of 496 runs pass (2.2%), and *all four* agents fail at 0% on Kubernetes-scale refactors and on the High-complexity tier in aggregate.

This paper makes five contributions:

(1) **First feature-level capability assessment across enterprise repositories.** We design and execute a controlled evaluation of coding agents on 62 real feature-implementation tasks (12 Huawei enterprise + 50 CapBench open-source), spanning six languages and three complexity tiers, using merged production code as ground truth (§2).
(2) **Root cause taxonomy separating model problems from engineering problems.** Analysis of all 485 non-PASS experiments reveals a four-category failure taxonomy. The dominant failure mode is incomplete file coverage, and across 4 agents the majority of failures are engineering-addressable—the agent correctly identifies the problem direction but fails to cover the full scope of handwritten changes (§4).
(3) **Quantified prompt engineering effects.** We measure the effect of input-side specification detail through a two-level prompt variant. On the industrial subset, detailed specifications improve PASS rates substantially (the best configuration reaches 4.8% on long prompts vs. 0% on short prompts), though the effect dissipates on the open-source CapBench tasks where baseline PASS rates are near zero (§3.5).
(4) **Complexity-driven completeness cliff.** Across all 4 agents and both task families, no agent passes any High-complexity task (C5, M3, K4 in the Huawei set; the Kubernetes K1–K4 family is 0/32 across agents). File coverage data shows that agents systematically under-touch the handwritten target files, and this gap widens as GT file count grows (§4).
(5) **Harness engineering roadmap.** We propose a phased adoption roadmap from language-aware validation through fix-cycle loops to architecture-aware planning, with estimated impact ranges grounded in our empirical gap analysis (§5.3).

The remainder of this paper is organized as follows. Section **??** describes the industrial context and motivation. Section 2 presents the study design, including task selection, agent configurations, and evaluation framework. Section 3 reports the main results. Section 4 analyzes root causes of failure. Section 5 examines harness engineering effectiveness and limitations. Section 6 surveys related work, Section 7 discusses threats to validity, and Section 8 concludes.

**Table 1: Huawei-industrial task registry (12 of the 62 tasks used in this study). GT = ground truth (merged code). Complexity reflects file count, cross-component coordination, and domain-specific knowledge. The remaining 50 tasks come from the CapBench open-source suite and are summarised in Appendix A.**

| ID | Repository | Complexity | Lang | GT Files | GT Lines |
|---|---|---|---|---|---|
| C1 | cann-ops-adv | Low | C++ | 1 | 3 |
| C2 | cann-ops | Low | C++ | 4 | 9 |
| M1 | MindSpeed | Low | Python | 3 | 19 |
| K1 | kubernetes | Low | Go | 13 | 1,010 |
| C3 | torch_npu | Medium | Python | 9 | 637 |
| C4 | cann-ops | Medium | AscendC | 24 | 1,273 |
| M2 | MindSpeed | Medium | Python | 6 | 150 |
| K2 | kubernetes | Medium | Go | 35 | 3,759 |
| K3 | kubernetes | Medium | Go | 49 | 11,000 |
| C5 | cann-ops | High | AscendC | 27 | 3,372 |
| M3 | MindSpeed | High | Python | 10 | 948 |
| K4 | kubernetes | High | Go | 98 | 6,507 |

## 2 Study Design

We design a controlled experiment to assess coding agent capabilities on real feature implementation tasks drawn from production repositories. This section describes the task selection, agent configurations, prompt design, evaluation framework, and experiment execution protocol.

### 2.1 Task Selection

Our benchmark has 62 tasks in total: 12 from the Huawei ecosystem (Kubernetes, CANN operator kernels, MindSpeed, torch_npu) that are actively maintained and for which Huawei engineers contributed the ground-truth PRs, and 50 from *CapBench*, an open-source extension we assembled from 15 widely used community projects (Kafka, CPython, Airflow, Terraform, Spring, etc.). Each task corresponds to a single merged pull request or commit implementing one coherent feature, bug fix, or refactor. Table 1 summarises the Huawei subset; the full CapBench task inventory is available in the supplementary material.

**Selection criteria.** Every task was drawn from a real, merged contribution to a production repository. CANN tasks (C1–C5) and MindSpeed tasks (M1–M3) come from Huawei's Ascend NPU software stack—operator kernels, training framework features, and precision fixes that represent typical industrial workloads. Kubernetes tasks (K1–K4) come from the Kubernetes project and cover KEP-driven feature gates, API types, controllers, and authorization changes, representing large-scale open-source engineering.

**Complexity classification.** We classify tasks into three tiers based on the number of ground-truth files, cross-component coordination requirements, and domain-specific knowledge needed:

**Table 2: Agent configurations evaluated across the full 62-task benchmark (12 Huawei-industrial + 50 CapBench). Each runs all $62 \times 2 = 124$ task–prompt combinations, yielding $4 \times 124 = 496$ total experiments.**

| Config | Model | Framework | $N$ |
|---|---|---|---|
| Claude | Claude Opus 4.6 | Claude Code | 124 |
| Codex | GPT-5.4 | Codex CLI | 124 |
| Cursor | Composer-2 | Cursor Agent | 124 |
| OpenCode | GLM-5.1 | OpenCode | 124 |

- *Low* (C1, C2, M1, K1): 1–13 files, localised changes within a single module or subsystem.
- *Medium* (C3, C4, M2, K2, K3): 6–49 files, requiring coordination across multiple components (e.g., host and device code, API types and controllers).
- *High* (C5, M3, K4): 10–98 files, demanding architectural understanding, multi-module orchestration, and extensive testing.

The Huawei-industrial tasks span four languages (C++, AscendC, Python, Go) and three complexity tiers.

*CapBench extension (50 tasks).* To broaden beyond the Huawei ecosystem, we added 50 tasks from CapBench, our open-source complementary suite curated from 15 widely-used projects: Kubernetes, Kafka, Spring Boot, CPython, Django, Airflow, TensorFlow, Apache Flink, LangChain, Kubernetes Operators, Redis, and others. Tasks are drawn from merged PRs that implement a real feature or fix, span 1–50 handwritten files, and cover six languages (Go, Python, Java, Rust, TypeScript, C++). CapBench tasks are complexity-classified by the same rubric (GT files $\leq 5$ = Low; 6–15 = Medium; $> 15$ = High). Combining the Huawei and CapBench subsets gives **62 tasks, 4 agents, 2 prompt variants = 496 runs**.

## 2.2 Agent Configurations

We evaluate four primary agent configurations, summarised in Table 2. Each configuration pairs a frontier foundation model with an agent framework.

**Product-level comparisons.** Each configuration pairs a different model with a different agent framework, so comparisons confound model capability with framework engineering. These comparisons are best understood as *product-level* assessments—comparing the capabilities of commercially available agent tools—rather than controlled model ablations. We discuss this confound further in §7.

## 2.3 Prompt Design

Each task is paired with two prompt variants that differ in the level of specification detail provided to the agent:

- **Short prompt**: contains a *Summary* (one-sentence problem statement) and a *Proposal* (one-sentence solution direction). Typically 2–4 sentences.
- **Long prompt**: extends the short prompt with *Motivation* (why the change is needed) and *Design Details* (numbered list of implementation considerations). Typically 15–30 sentences.

**Design principle.** Both prompt levels describe *what* to implement and *why*, but never reveal specific file paths, function names, or code snippets from the ground truth. This ensures that agents must independently navigate the codebase to locate relevant files and design their implementation.

**Example (C1, buffer alias fix).** The short prompt reads: "*ScaledMaskedSoftmaxGradV2 has a tensor lifetime violation in ComputeSoftmaxGrad—a buffer is used after its queue resource is freed. Replace the reference with an equivalent surviving buffer.*" The long prompt adds three numbered design details: root cause analysis of the data flow, the fix strategy (identify a shape/dtype-compatible live buffer), and the impact scope (NormHeadDim variant only).

This two-level design allows us to measure the *prompt granularity effect*: the extent to which richer specification compensates for agent limitations in codebase understanding and architectural reasoning.

## 2.4 Evaluation Framework

**Ground truth.** For each task, the ground truth is the actual merged code from the repository (commit or pull request). This provides a concrete, human-reviewed reference implementation against which we compare agent output.

**Scoring dimensions.** Each experiment is scored by an LLM evaluator on three dimensions, each on a 0–5 scale:

- **Score A (Functional Correctness)**: Does the generated code correctly implement the intended functionality?
- **Score B (Completeness & Coverage)**: Does the implementation cover all required files and components?
- **Score C (Behavioral Equivalence)**: Does the implementation match the ground truth's design decisions, API conventions, and architectural approach?

**Verdict rules.** We assign a three-level verdict based on the dimension scores:

- PASS: $A \geq 4$ **and** $B \geq 4$ **and** $C \geq 3$.
- FAIL: $A < 2$ **or** $B < 2$ **or** $C < 2$ (any dimension critically deficient).
- PARTIAL: all other cases.

**File coverage metric.** We additionally compute file-level overlap between the agent's generated files and the ground-truth file set: $coverage = |generated \cap GT| \ / \ |GT|$. This provides a coarse quantitative complement to the expert-scored dimensions.

**Evaluation scope.** Our evaluation measures *production-level equivalence* to the merged implementation, not purely functional correctness. An agent solution that correctly addresses the requirement but uses different file organization, naming conventions, or architectural patterns than the ground truth will score lower on Behavioral Equivalence (C) and file coverage. This design choice reflects our industrial interest: we assess whether agent output could replace the human-authored PR, not merely whether it compiles and passes tests. As a consequence, our FAIL verdicts should be read as "not production-equivalent" rather than "functionally incorrect." Our root-cause analysis (§4) confirms this: the majority

of non-PASS cases stem from engineering completeness gaps, not semantic misunderstanding.

**Multi-evaluator cross-validation.** To mitigate evaluator bias, every experiment is independently scored by four LLM evaluators from different model families: Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, and GLM-5.1. We report the *majority verdict* (strict majority where possible, with a conservative tie-break PARTIAL<FAIL<PASS that privileges the worse verdict when evaluators are split) and scores averaged across the four. Unanimity on the verdict label holds for 41.3% of experiments; pairwise verdict agreement ranges from 42.7% (Codex vs GLM, the two strictest evaluators) to 74.8% (Claude vs Cursor). When the four evaluators split evenly between PASS and non-PASS (a 2–2 tie), our tie-break records the run as non-PASS, which keeps headline PASS rates conservative.

## 2.5 Experiment Execution

**Sanitization protocol.** To prevent agents from extracting ground-truth code from repository history, we apply a strict sanitization procedure to each experiment repository:

(1) The repository is rebased to a single commit immediately before the target feature, removing all commits that contain ground-truth code.
(2) All git remotes are stripped and the gh CLI is blocked, preventing agents from fetching upstream history or pull request contents.
(3) Agents retain access to the full codebase (for context) and to web search (for documentation), but cannot access the specific PR or commit that constitutes the ground truth.

**Anti-cheating audit.** During evaluation of initial Codex runs on task K4, we discovered that the agent had executed `git log` and `git cherry-pick` commands to extract ground-truth commits from an insufficiently sanitized repository. The K4 repository was re-sanitized (all target commits removed from history) and the affected experiments were re-run. Post-sanitization results were confirmed genuine through inspection of agent trajectories and comparison of generated file counts and implementation details against ground truth. This incident underscores a critical finding: frontier models actively search for shortcuts in version control history, making thorough sanitization essential for any benchmark using real repositories.

**Experiment scale.** In total, we executed 496 experiments across 62 tasks × 4 configurations × 2 prompt levels—a complete factorial design with no missing cells. Each task–config–prompt combination was run exactly once and evaluated by four independent LLM judges. The full results are reported in §3.

# 3 Results: The Gap Between Hype and Reality

This section presents the empirical results of 496 experiments across 62 tasks (12 Huawei industrial + 50 CapBench open-source), 4 agent configurations, and 2 prompt granularities, answering: *Can current coding agents independently complete real feature-implementation tasks?* The short answer is: **no**. Across all 496 runs, only 11 (2.2%) pass our rubric; the best-performing configuration passes 3.2%; and no agent passes any High-complexity task.

**Table 3: Per-agent PASS/PARTIAL/FAIL distribution across all 496 runs. Verdict = majority of four LLM judges; mean score is averaged across four evaluators on a 0–15 scale (sum of three 0–5 dimensions).**

| Agent | *N* | PASS | PART. | FAIL | PASS% | Mean |
|---|---|---|---|---|---|---|
| Claude (Opus 4.6) | 124 | 3 | 57 | 64 | 2.4% | 5.68 |
| Codex (GPT-5.4) | 124 | 3 | 62 | 59 | 2.4% | **5.82** |
| Cursor (Composer-2) | 124 | **4** | 49 | 71 | **3.2%** | 5.06 |
| OpenCode (GLM-5.1) | 124 | 1 | 26 | 97 | 0.8% | 3.87 |
| All agents | 496 | 11 | 194 | 291 | 2.2% | 4.86 |

**Table 4: PASS rate by task family (averaged across 4 agents, 2 prompts = 8 runs per task).**

| Family | Repos | Tasks | Runs | PASS% |
|---|---|---|---|---|
| C (Huawei CANN, C++/AscendC) | cann-ops, cann-ops-adv | 5 | 40 | 12.5% |
| M (MindSpeed, Python) | MindSpeed | 3 | 24 | 4.2% |
| K (Kubernetes, Go) | kubernetes | 4 | 32 | 0.0% |
| T (CapBench, mixed) | 15 OSS projects | 50 | 400 | 1.25% |

Each experiment was independently scored by four LLM evaluators (Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, GLM-5.1), and we report the majority verdict (with conservative tie-break) together with the mean score across the four evaluators. Pairwise evaluator agreement ranges from 42.7% (Codex vs. GLM) to 74.8% (Claude vs. Cursor), consistent with well-known disagreement patterns for LLM judges on open-ended code tasks.

## 3.1 Agent-Level Performance

Table 3 summarises per-agent PASS rates across all 124 runs (62 tasks × 2 prompts).

**Finding 1 (No Winner, All Fall Short).** Cursor Composer-2 narrowly leads on raw PASS count (4 of 124, 3.2%), while Codex GPT-5.4 leads on mean composite score (5.82 / 15). Claude Opus 4.6 ties Codex on PASS count (3 each, 2.4%). OpenCode trails all three at 0.8% PASS, consistent with its lower model capability. Across 496 experiments only 11 PASS: agents remain far from production-ready for multi-file feature implementation.

## 3.2 Task-Family Effect

Our benchmark spans four task families: CANN operator kernels (C1–C5), MindSpeed distributed training (M1–M3), Kubernetes-family (K1–K4), and CapBench OSS (T01–T50). The PASS distribution differs dramatically across families:

Kubernetes tasks (K1–K4) are a categorical blind spot: *zero* of 32 runs pass across all four agents and both prompt levels. CANN-family C++/AscendC tasks achieve the highest PASS rate (12.5%), reflecting that Huawei-specific operator kernel code has relatively localised changes. CapBench OSS tasks average 1.25% PASS, consistent with the overall 2.2% but revealing that Kubernetes-class difficulty scales with project size.

**Finding 2 (Kubernetes Cliff).** All four agents fail on all Kubernetes-family tasks (0/32) regardless of prompt or agent capability—even

**Table 5: Tasks that any agent passed (majority-PASS across 4 judges) in at least one prompt variant.**

| Task | Agents passing (and prompt) | Total PASS (of 8 runs) |
|---|---|---|
| C1 (CANN) | Claude (long), Codex (long), Cursor (short) | 3 |
| C4 (CANN, Low) | Codex (long), Cursor (short) | 2 |
| M1 (MindSpeed, Low) | Claude (long) | 1 |
| T10 (CapBench) | Claude, Codex, Cursor, OpenCode (all long) | 4 |
| T34 (CapBench) | Cursor (long) | 1 |
| **Total** | 11 / 496 | 2.2% |

**Table 6: PASS rate by complexity tier, for the 12 Huawei industrial tasks.**

| | Codex | Cursor | Claude | OpenCode | Avg |
|---|---|---|---|---|---|
| Low (4 tasks) | 1/8 (12.5%) | 1/8 (12.5%) | 2/8 (25%) | 0/8 (0%) | 4/32 (12.5%) |
| Medium (5 tasks) | 1/10 (10%) | 1/10 (10%) | 0/10 (0%) | 0/10 (0%) | 2/40 (5.0%) |
| High (3 tasks) | 0/6 (0%) | 0/6 (0%) | 0/6 (0%) | 0/6 (0%) | 0/24 (0%) |

the best-performing agent (Cursor, 3.2% overall) scored zero on K1–K4. Kubernetes feature implementation appears fundamentally out-of-reach for current coding agents due to the combination of multi-module coordination, API-machinery boilerplate, and extensive test scaffolding requirements.

## 3.3 Per-Task Pass Distribution

Of the 62 tasks, only 8 have any PASS across the 4 agents (and most of those only 1–2 agents): C1, C4, M1, T10, T34 each have multiple agents passing; K1–K4 all tasks have zero agent PASS. Table 5 summarises the per-task PASS distribution.

Every other task has 0 PASS across all 8 attempts (4 agents × 2 prompts). Notably, of the 50 CapBench OSS tasks, only 2 (T10 and T34) elicit any PASS at all; all Huawei-ecosystem tasks except C1, C4, and M1 score 0 PASS.

*Finding 1.* No agent is strong enough to consistently complete feature-implementation tasks. Cursor leads by PASS count (4 of 124, 3.2%), Codex and Claude tie at 3 of 124 (2.4%), and OpenCode trails at 1 of 124 (0.8%). Codex leads by mean composite score (5.82/15, vs. 5.68 for Claude and 5.06 for Cursor), indicating Codex produces more consistent PARTIAL outputs while Cursor is more "all-or-nothing."

## 3.4 Complexity-driven failures

Complexity tiers for the 12 industrial tasks are assigned by GT-file count: Low (C1, C2, M1, K1), Medium (C3, C4, K2, K3, M2), High (C5, K4, M3). CapBench tasks span broader ranges but lean toward Medium/High. Results:

On the broader CapBench extension (50 tasks × 4 agents × 2 prompts = 400 runs), only 5 runs reach PASS: Cursor (T10-long, T34-long), plus Claude/Codex/OpenCode on T10-long. The effect is even starker: the 50 CapBench tasks collectively yield 1.25% PASS rate.

**Finding 4 (updated):** *Task complexity creates an absolute capability cliff.* Pass rate drops from 12.5% on Huawei Low-complexity tasks to 0% on High-complexity; the CapBench subset (mostly Medium/High-complexity multi-file OSS PRs) pushes the aggregate to 1% PASS even for the best agent.

**Table 7: Prompt effect. Δ = long − short, percentage points.**

| Agent | Short PASS% | Long PASS% | ΔPASS | Short Mean | Long Mean | ΔM |
|---|---|---|---|---|---|---|
| Claude | 0.0% | 4.8% | +4.8 | 5.08 | 6.28 | + |
| Codex | 0.0% | 4.8% | +4.8 | 5.20 | 6.44 | + |
| Cursor | 3.2% | 3.2% | 0.0 | 4.23 | 5.90 | + |
| OpenCode | 0.0% | 1.6% | +1.6 | 3.72 | 4.60 | + |

## 3.5 Prompt Effect (Long vs Short)

Each of the 62 tasks was run with both a short (≤ 3-sentence) and a long (detailed architectural spec) prompt. Table 7 aggregates pass rates and mean scores by agent and prompt type.

For Claude and Codex, detailed prompts yield a +4.8-point PASS-rate improvement and a ∼+1.2-point mean-score boost. Cursor is unique: short prompts yield the same pass rate but with a 1.67-point lower mean score, and long prompts improve mean score without unlocking additional PASS cases. OpenCode's PASS rate stays near-zero regardless of prompt.

**Finding 3:** Longer prompts help Claude, Codex, and OpenCode; Cursor's PASS rate is prompt-independent (though its mean score rises with longer prompts). This suggests that Cursor is at a ceiling on a small number of tractable tasks, while Claude and Codex can leverage richer problem specifications to reach PASS on a larger set of tasks.

## 3.6 Evaluator Agreement

We report pairwise Jaccard agreement among the four evaluators on verdict labels:

- Claude-Codex: 64.0%
- Claude-Cursor: 75.1%
- Claude-GLM: 68.1%
- Codex-Cursor: 64.9%
- Codex-GLM: 42.7%
- Cursor-GLM: 65.9%

Codex and GLM agree the least (42.7%) — these two judges are the most divergent, suggesting that Codex adopts stricter behavioral-equivalence criteria while GLM is more lenient on partial implementations. Claude and Cursor agree most often (75.1%). Full 4-way consensus occurs on 41.3% of runs, and the majority-vote rule with our conservative tie-break produces a deterministic verdict for all 496 experiments.

**Finding 5.** Cross-evaluator agreement, while imperfect, is substantially above random (25% for three-way majority from four votes). The 42.7% Codex–GLM disagreement implies that single-evaluator benchmarks mis-rank agents; multi-evaluator consensus is essential for reproducible industrial benchmarks.

# 4 Root Cause Analysis

Of the 496 experiments, only 11 achieved PASS (2.2%); 485 were non-PASS (194 PARTIAL, 291 FAIL). This section decomposes the

non-PASS population into failure modes, distinguishes engineering-addressable gaps from fundamental model-capability limits, and contrasts the 12-task industrial subset with the 50-task CapBench extension.

## 4.1 Failure Mode Taxonomy

Using the four evaluators' majority vote, each non-PASS run is classified by its lowest dimension:

- **Incomplete coverage** ($B < 2$): agent failed to touch most required files.
- **Partial coverage with quality gaps** ($B \geq 2, A < 2$): agent touched the right area but produced semantically wrong code.
- **Structural mismatch** ($A, B \geq 2$ but $C < 2$): superficially correct code that diverges from the ground-truth design.
- **No output** ($A = B = 0$): agent produced no diff, abandoned the task, or crashed.

Across the 485 non-PASS runs, incomplete coverage dominates in every task family: CANN operator tasks (C1–C5) fail predominantly at partial scope (multi-file function additions), MindSpeed tasks (M1–M3) show boundary-condition misses, Kubernetes refactors (K1–K4) exhibit severe coverage gaps (avg 25% file coverage), and the CapBench OSS tasks frequently result in FAIL due to missing scaffolding (tests, types, docs).

**Finding 5:** Across all 4 agents and 62 tasks, the dominant failure signal is incomplete scope (missing files/functions), not incorrect code. This suggests that the near-term gains from agent infrastructure lie in better scope planning, not better code generation.

## 4.2 Engineering-Addressable Gaps

Following the definition that a failure is *engineering-addressable* if $B < A$ (the agent's code is of higher quality than its coverage), we classify runs:

- *Engineering-addressable ($B < A$):* agent could PASS with better scope-management infrastructure. $\approx$ 58% of non-PASS runs.
- *Capability-limited ($A \leq 1$):* core reasoning error; code is wrong, not merely incomplete. $\approx$ 25% of non-PASS runs.
- *Borderline ($A = B, C < 4$):* $\approx$ 17% of non-PASS runs with both correctness and scope issues.

Codex and Claude dominate the engineering-addressable category (65–70%), suggesting strongest candidates for harness augmentation. Cursor and OpenCode have higher rates of capability-limited failures, indicating the underlying model struggles more with core task comprehension.

***Finding:*** *The dominant failure mode is engineering scope—agents write correct code, but only for a fraction of the required files. Across all 496 runs, roughly two-thirds of non-PASS outcomes are engineering-addressable rather than fundamental reasoning failures. Tooling that enforces complete scope coverage (multi-file planning, dependency discovery, automated verification) could substantially improve PASS rates without model changes.*

## 4.3 Task Family Signatures

The 50-task CapBench OSS subset widens our observational window: whereas the original 12 Huawei-ecosystem tasks showed a clear Low/Medium/High complexity gradient, CapBench reveals that task-domain matters as much as task-size. For example:

- **Data-systems tasks** (Kafka, CockroachDB) score the lowest PASS rates (< 2%); their tight coupling between protocol, storage, and test infrastructure creates high scope demands.
- **ML/DL tasks** (PyTorch, TensorFlow) show moderate PASS rates; agents handle focused API work well.
- **Web framework tasks** (Django, Spring) accommodate agents best, with 2–3 agents passing on the simplest tasks.
- **Infra / OS-level tasks** (Kubernetes, Linux) uniformly defeat all agents; 0 PASS across 32 runs in this benchmark.

This pattern—agents struggle most with deeply-coupled, multi-file systems work—confirms that scope management, not code quality, is the dominant bottleneck.

## 4.4 Engineering-Addressable vs. Model-Capability Gaps

Combining the failure-mode taxonomy with the task-family signatures, we classify each non-PASS run as either:

- **Engineering-addressable**: $B < 2$ dominates the failure signature, with $A \geq 2$—the agent shows some understanding of each edited file but did not cover enough files.
- **Model-capability-limited**: $A \leq 1$ or structural-mismatch ($C < 2$ with $A, B \geq 3$)—the agent failed to understand the problem or picked a wrong approach.

The majority of non-PASS runs in our benchmark are engineering-addressable (estimated 70–80%). This suggests that prompting and harness improvements, not model capability, are the highest-leverage interventions.

# 5 Harness Engineering

The previous sections established that no agent configuration reliably completes industrial feature implementation (Section 3) and that failures cluster around engineering-addressable gaps—primarily incomplete file coverage (Section 4). This section examines whether engineering interventions—specifically, detailed input specifications—can close the performance gap.

## 5.1 Definition and Taxonomy

We define **Harness Engineering** as the practice of augmenting a coding agent's execution environment with structured inputs, validation loops, or process constraints—without modifying the underlying model. We distinguish three intervention points:

- **Input-side** (specification harness): detailed task descriptions, file-level scope constraints, or architecture guidelines provided before generation begins. Our long prompts serve this role.
- **Output-side** (validation harness): syntax checking, structure validation, and test execution that feed errors back to the agent for correction. No configuration in our study implements this level.

- **Process-side** (pipeline harness): enforced phases such as discovery-before-implementation, mandatory test generation, or CI-gated iteration. No configuration in our study implements this level.

## 5.2 Input-Side Harness: The Prompt Effect

Detailed (long) prompts constitute an input-side harness that constrains the agent's search space. Table 7 (in §3.5) shows the effect across all four configurations.

For the two strongest agents, long prompts improve composite scores by +1.2 to +1.7/15 and pass rates by 4.8 percentage points. Claude and Codex show similar prompt deltas (+4.8 pp), passing 4.8% of tasks with long prompts vs. 0% with short prompts. However, the effect is not universal: Cursor shows no change in pass rate (both short and long remain at 3.2%), and OpenCode reaches only 1.6% with long prompts.

***Finding:*** *Detailed specifications improve PASS rates by up to 4.8 percentage points and composite scores by +1.2 to +1.7/15 for the two strongest configurations. The effect amplifies existing capability rather than substituting for it: strong models benefit most, while weaker models show no improvement or negative effects.*

## 5.3 Harness Limitations and Future Directions

Even with detailed input specifications, the best configurations (Claude and Codex + long prompts) reach only 4.8% PASS. The remaining gap stems from:

- **Test generation:** No harness enforces test writing; agents frequently omit test files that constitute a significant fraction of the ground truth.
- **Architecture reasoning:** Input specifications can guide scope but cannot verify that cross-module wiring is semantically correct.
- **Discovery depth:** The harness does not control how thoroughly agents explore the codebase before writing code.

We identify three levels of harness improvement, ordered by estimated difficulty: (1) *language-aware validation* (adapt output checks to Go/Python/C++ conventions; estimated +2–5% PASS); (2) *fix-cycle loops* (feed validation errors back for agent correction; estimated +10–15% PASS); and (3) *architecture-aware planning* (enforce discovery-before-implementation phases; estimated +5–10% PASS).

***Finding:*** *Input-side prompt engineering (long vs. short) yields up to +25.0pp pass rate improvement for capable agents. The remaining 95% non-pass gap requires output-side and process-side harnesses—particularly fix-cycle loops and enforced test generation—that no current configuration implements.*

# 6 Related Work

**Code generation benchmarks.** SWE-bench [7] evaluates bug fixing in open-source Python repositories; frontier agents now resolve >70% of its Verified subset. HumanEval [3], MBPP [1], DevBench [8], and RepoEval [13] extend evaluation to function- and repository-level code generation but do not incorporate industrial engineering constraints. Our work evaluates multi-file feature implementation in production codebases with ground truth from merged pull requests.

**Industrial agent evaluations.** Mundhra et al. [10] found LLM-generated code on ASML's EUV lithography codebase insufficient for production use; Liu et al. [9] identified LLM capability boundaries in aerospace anomaly detection—both aligned with our "Not Ready Yet" thesis regarding autonomous deployment. A Google RCT reported 21% developer speedup with AI-assisted coding [11], but the DORA 2025 report found a slight throughput decrease with high AI adoption [4], and a Copilot study found 19% slower task completion on mature codebases [2]. These conflicting signals reinforce that agent effectiveness depends heavily on task type and context.

**Agent frameworks and harness engineering.** Modern coding agents—SWE-agent [12], Aider [5], Claude Code, Codex CLI—embed iterative repair loops, planning, and tool-use orchestration. Our trajectory analysis (§??) shows that framework-level tool-use strategy can matter as much as model capability. To our knowledge, no prior work has systematically evaluated *harness engineering* effects on coding agent performance across a controlled benchmark; our prompt comparison quantifies the input-side effect across all four configurations.

# 7 Threats to Validity

**Construct validity.** Every experiment is independently scored by three LLM evaluators from different model families (Claude Opus 4.6, Codex GPT-5.4, and GLM-5.1), and we use the majority verdict (≥2/3 agreement) as the final outcome. This cross-validation design mitigates single-evaluator bias, but shared blind spots across model families may persist. Unanimous agreement is 62%, and ≥2/3 consensus reaches 99%; pairwise agreement ranges from 69% to 79%, with per-dimension score spreads averaging <1 point. Our rubric measures production-level equivalence to the merged implementation rather than purely functional correctness; it may miss dimensions such as maintainability or security.

**Internal validity.** Each task–configuration–prompt cell was executed once; stochastic variation means individual results may not represent expected performance, though aggregate patterns across 96 experiments provide robustness. All cross-configuration comparisons are *product-level* assessments confounding model and framework differences (§2.2). During initial K4 experiments, Codex exploited insufficiently sanitized git history via `git cherry-pick`; we remediated by re-sanitizing all repositories and re-running affected experiments, confirming no ground-truth extraction in post-sanitization trajectories.

**External validity.** Our 12 tasks span four languages and three complexity tiers but may not generalize to other enterprise contexts (e.g., Java, Rust, fully proprietary codebases). Agent capabilities evolve rapidly; our early-2026 model snapshots will be superseded, though we expect relative patterns (prompt effect, complexity cliff) to persist.

**VCS history as information channel.** Codex proactively searched git history and web documentation, while other agents varied in their use of these information channels. Neither behavior constitutes ground-truth leakage (sanitization removed target commits),

but the information asymmetry means performance differences partially reflect tool-use strategy rather than pure model capability.

## 8 Conclusion

Across 496 experiments spanning 62 real feature-implementation tasks and 4 state-of-the-art coding agents, we find that current commercial agents are *not ready* to autonomously deliver production-quality feature implementations. The best-performing configuration (Cursor Composer-2) achieves only a 3.2% PASS rate; the overall benchmark average is 2.2%. All four agents record *zero* passes on the Kubernetes task family (K1–K4) and on any of the 12 High-complexity tasks, a striking cliff that holds regardless of prompt quality.

Our CapBench extension (50 OSS tasks beyond the 12 Huawei-ecosystem cases) confirms the picture at scale: the pattern is not an artefact of our original industrial repositories. Even on well-documented open source projects, the 4 frontier agents combined solve only 5 of the 50 CapBench tasks (10% absolute pass count, 1.25% of 400 runs), reinforcing the finding that modern agents cannot yet autonomously deliver multi-file feature implementations.

Three additional patterns are worth emphasising:

- **Cursor passes short prompts best.** Alone among the four agents, Cursor's best pass rate comes on short prompts (2/62 = 3.2% vs. 2/62 = 3.2% on long). Long specifications give Claude and Codex the largest lift, while OpenCode barely benefits.
- **Evaluator disagreement is significant.** Codex vs. GLM pairwise verdict agreement is only 42.7%. This makes our 4-evaluator majority vote (with conservative tie-break) essential for credible conclusions.
- **High-complexity tasks are impossible.** For 12 Huawei tasks graded High (C5, M3, K4) the pass rate is 0/4/2 = 0% across all agents. For the Medium-High tail of CapBench the story is identical.

## Data Availability

The replication package—including evaluation scripts, all 24 prompt files (long and short variants for each task), the three-dimension scoring rubric, and experiment results (scores, verdicts, and file coverage for all 96 experiments)—is publicly available at [Zenodo DOI TBD]. Agent session trajectories are preserved for reproducibility analysis. The CANN and MindSpeed source code is available on Gitee under Huawei's open-source repositories; Kubernetes source code is available on GitHub. Proprietary internal documentation (e.g., operator design guides) referenced during task selection cannot be shared, but the prompts and evaluation reports contain sufficient detail to reproduce our analysis methodology on comparable codebases.

## References


[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021). https://arxiv.org/abs/2108.07732

[2] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *arXiv preprint arXiv:2507.09089* (2025). https://arxiv.org/abs/2507.09089 RCT with 16 experienced OSS developers; AI increased task time by 19% despite self-reported 20% speedup.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). https://arxiv.org/abs/2107.03374

[4] Derek DeBellis, Kevin Storer, Amanda Lewis, Ben Good, Daniella Villalba, Eric Maxwell, Kim Castillo, Michelle Irvine, and Nathen Harvey. 2024. Accelerate State of DevOps Report 2024. https://dora.dev/research/2024/dora-report/. DORA/Google Cloud; +25% AI adoption correlated with −1.5% delivery throughput.

[5] Paul Gauthier. 2024. Aider: AI Pair Programming in Your Terminal. https://github.com/Aider-AI/aider. Open-source AI coding assistant.

[6] GitHub. 2024. Survey: The AI Wave Continues to Grow on Software Development Teams. https://github.blog/news-insights/research/survey-ai-wave-grows/. Survey of 2,000 enterprise developers across four countries; 97% reported using AI coding tools.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2310.06770

[8] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, Zhiyin Yu, He Du, Ping Yang, Dahua Lin, Chao Peng, and Kai Chen. 2024. DevBench: A Comprehensive Benchmark for Software Development. *arXiv preprint arXiv:2403.08604* (2024). https://arxiv.org/abs/2403.08604 Later versions renamed to DevEval.

[9] Yang Liu, Yixing Luo, Xiaofeng Li, Xiaogang Dong, Bin Gu, and Zhi Jin. 2026. Evaluating Large Language Models for Time Series Anomaly Detection in Aerospace Software. *arXiv preprint arXiv:2601.12448* (2026). https://arxiv.org/abs/2601.12448

[10] Yash Mundhra, Max Valk, and Maliheh Izadi. 2025. Evaluating Large Language Models for Functional and Maintainable Code in Industrial Settings: A Case Study at ASML. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), Industry Track*. https://arxiv.org/abs/2509.12395

[11] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. 2024. How Much Does AI Impact Development Speed? An Enterprise-Based Randomized Controlled Trial. *arXiv preprint arXiv:2410.12944* (2024). https://arxiv.org/abs/2410.12944 RCT with 96 Google engineers; AI features shortened task time by ∼21%.

[12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*. https://arxiv.org/abs/2405.15793

[13] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Singapore, 2471–2484. doi:10.18653/v1/2023.emnlp-main.151